Worlds, Not Windows

Spatially Aware Creative Agentic Interfaces

Harrison Edwards

Omo Research

DRAFT v1.8, not for circulation. A friend's consent to be quoted must be confirmed before release (§4.5).



Figure 1. Worlds, not windows. An organisation drawn as an open world of districts, with agents at work in each and all of them linked to a central status wall. *Generated illustration* (Omo Space enterprise site), not a screenshot; captures from Minecraft appear in Figures 3–5.

Abstract. AI agents now run for hours, many at a time, yet we still manage them through chat windows and terminal tabs. With twenty agents running, the tab bar stops telling you which agent is which and which one needs you. We tried something else: we gave each agent session a place in a Minecraft world. Omo Space is a mod that puts real agent sessions (Hermes Agent, Claude Code) and live web pages on in-world screens, with lamps that show when an agent is working. The sessions themselves keep running on the user's computer, and the places are mostly built by coding agents from short requests. This paper shares findings so far from the builder using it himself. He worked in it every day while streaming (25–29 October 2026), and a friend built his own world with it. Looking back over about 29 hours of transcribed streams, we describe four ways place got used: to *find* work by walking to it, to *resume* by coming back to it, to *monitor* by glancing at lit screens, and to *orchestrate* a pipeline by moving work from place to place. We also report his real usage from the agents' own logs: about 7 billion tokens over the five days, and over 4 billion on 28 October alone. We have not tested it against tabs; we describe what it was like, what it suggests, and what we would like to measure next.

Keywords: agentic interfaces; spatial interfaces; persistence; ambient awareness; multi-agent supervision; generative environments; Minecraft; autobiographical design

1 Introduction

AI agents have gone from one chat to many long-running sessions: coding agents, research agents, video-editing agents. They work for hours, wait for approval and ask questions. Most of us still run them in chat windows or terminal tabs. That works for one agent. With twenty, the tab bar no longer tells you which session is which, which one is busy, and which one is waiting for you.

The builder of the system in this paper put the motivation this way, on the first night of streaming it:

> *"So it's like I'm spatially aware of my workspace. … It's not a chat window, it's an interface."* [P1, T1, 00:38:33–00:38:50, live]

Using space to organise computer work is an old idea. Rooms (1986) split a crowded screen into virtual rooms [1], Data Mountain (1998) showed people remember where they put things in 3D [2], and the Task Gallery (2000) hung running applications on gallery walls [3]. Agents are a new kind of thing to organise. They keep working when you look away, they finish, and they ask for help. We wanted to know what a world of places does for that kind of work, and what it does not do.

> **What this paper is.** Findings so far, from one person building and using his own tool. Nothing here was set up as a test. The builder just used Omo Space for his real work, live on stream, and a friend tried it too. We went back through the recordings and wrote down what happened, in his own words, including what went wrong. We do not claim the world is faster than tabs; Section 7 lists what we would like to measure next.

We contribute:

1. **A system.** Omo Space, which shows live agent sessions and web pages as objects in a Minecraft world, with status lamps and agent-built places (§3).
2. **Findings so far.** Five days of the builder's own daily use, about 29 hours of stream transcripts, read with AI helpers and checked by script (§4).
3. **What we learned.** Four uses of place, the "web page with a terminal attached" pattern, places at every scale and a traced episode (§5).
4. **What to do next.** Design lessons and what we would like to measure next (§6–7).

2 Related Work

2.1 Space, memory and interruption

Malone found that piles on a desk do more than store things: "a very important function of desk organization is to remind the user of things to do" [4]. Rooms carried this into software with virtual workspaces, to cut "window thrashing" [1]. Data Mountain reported "statistically reliable advantages" for placing documents in space over a favourites list [2]. The Task Gallery, which hosted ordinary Windows applications in 3D, was found "enjoyable to use" [3]. People juggle many tasks and find switching hard [6], and they use spatial memory to return to "the general area where they were interrupted" [7]. Calm technology argues for displays that show state "without being obtrusive" [5]. All of this was about documents, windows and tasks; none of it was about supervising agents that keep working on their own.

2.2 Agents in game worlds

Minecraft is a common home for AI agents. Voyager explores and learns skills [8], Mindcraft studies agent teams and finds communication is a main bottleneck [10], and Generative Agents fills a game town with twenty-five agents people can watch and talk to [9]. In those systems the world is mostly where the agents act. In Omo Space the world is mainly the human's view of, and controls for, agents whose real work (code, video, publishing) happens outside the game. The line is not sharp, since Generative Agents is also something people watch.

2.3 Supervising many agents

The closest recent work tackles the same problem with dashboards. ParallelPilot adds planning, a run-logger and "an ambient dashboard alongside existing coding tools"; in a study with 16 people, participants "supervised an average of one more concurrent agent at peak" [19]. AgentGUI is built for "observing and steering AI agents amid multiple concurrent, long-running sessions" [20]. Magentic-UI offers "co-planning, co-tasking, multi-tasking, action guards, and long-term memory" [21]. Generative UI shows models can generate "not just the content, but the interface itself" [22]. Spatial hypertext found that "a visual/spatial metaphor … allows people to express the nuances of structure" [23]. Table 0 compares them. What Omo Space adds is a combination: places, built by people or agents, that hold long-running sessions and agent-made pages. Unlike ParallelPilot and AgentGUI it has not been tested with users. Unlike Magentic-UI it has no action guards. And unlike ParallelPilot's dashboard, which surfaces intervention cues, its lamps show activity but not "this agent is waiting for you".

Table 0. Omo Space and the nearest systems, as described in the material we inspected (mostly abstracts and introductions). "n/i" means not established from the inspected source; it does not mean the feature is absent.

System	Concurrent sessions	Where jobs run; persistence	State / intervention cues	Generated interface	Spatial organisation	Evaluation (task)
ParallelPilot [19]	Yes	Existing coding tools; n/i	Ambient dashboard; intervention cues	n/i	n/i	Within-subjects N=16 (short coding tickets)
AgentGUI [20]	Yes, long-running	Locally hosted; n/i	Trace visualisation; steering	n/i	n/i	Controlled study (finding key elements in traces)
Magentic-UI [21]	Multi-tasking	Web, code, files; n/i	Action guards; co-planning	n/i	n/i	Benchmarks, simulated users, qualitative study, safety tests
Generative UI [22]	n/i	n/i	n/i	Custom pages per prompt	n/i	Human preference ratings
VIKI [23]	n/i	n/i	n/i	n/i	User-arranged 2D space	n/i
Task Gallery [3]	Applications	Unmodified Windows apps	n/i	n/i	3D gallery (authored)	User studies
Omo Space	Yes	tmux on user's machine; survives client exit	Lamps, lit screens; no waiting state	Pages and places	Generated places, any scale	None yet (findings from own use)

2.4 How we looked back at it

Building a tool and learning from your own use of it has a name in HCI, autobiographical design: "design research drawing on extensive, genuine usage by those creating or building a system" [16], with an autoethnographic flavour [18]. We borrow the steps of thematic analysis [15]. Because AI models did the first reading, we call it an AI-assisted thematic synthesis rather than a full reflexive analysis. AI coding has matched experts

reasonably well when there is a fixed codebook [17], but ours was open-ended, so we also checked the outputs by script (§4.3).

3 System: Omo Space

Omo Space is a Fabric mod for Minecraft Java 26.2 plus a small local bridge [11][12]. We describe it from source snapshot `75d4677` (27 Oct 2026); the builds used day to day were not recorded. At the time it ran on Linux only [11].

3.1 What is in the world

A **workspace** is a named terminal session (tmux) on the user's computer. It has one or more **tabs**, each running an agent (Hermes, Claude Code, Codex) or a plain shell. Each Hermes agent has its own profile: memory, skills and model. In the world, a **station** is the computer block for a workspace. **Views** (wall monitors, slim screens) show a tab or a live web page, and several views can show the same workspace. A **place** is anything you build around them: a room, a cave, a tower, a world. Places are just builds; they carry no permissions and run nothing.

Table 1. In-world objects and their counterparts on the user's machine [11].

Object	Counterpart on the machine
Hermes Computer (station)	A named workspace whose first tab runs a Hermes agent in its own profile
Terminal	A workspace tab running a shell or another agent CLI
Monitor wall / Slim Screen	A larger view of a tab, or a pinned live web page (embedded Chromium)
Linker	Binds a view to a workspace without adjacency; several views may mirror one
Status Lamp	Follows an explicitly linked station, or else the nearest station within 8 blocks. Gold if any live tab in the workspace is busy, green otherwise, red once interrupted, dark without a live workspace
Name Plate	The workspace name, legible at a distance

A lamp sums up its workspace: gold if any tab is busy, otherwise green (source: `StationStatus.glow`). It has no "waiting for your answer" state. Agents do not pass work to each other through the world, and a friend loading the same map gets fresh workspaces of their own [11]. So a pipeline that runs across places is the user's own scripts and agent instructions, not something the mod does.

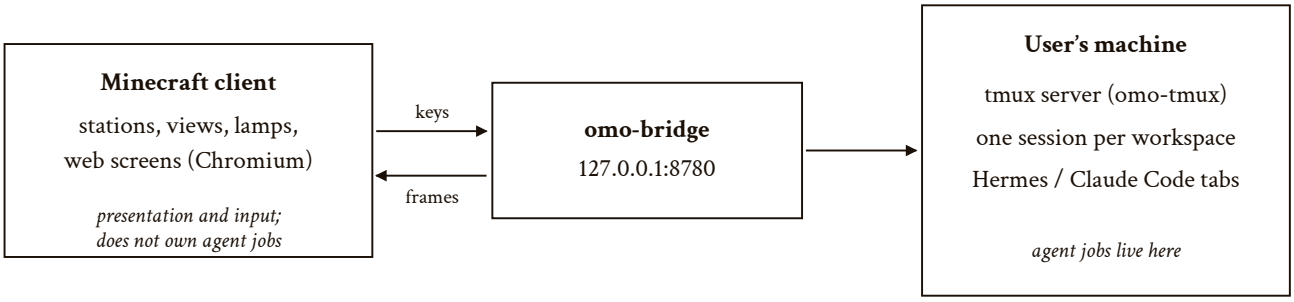


Figure 2. Architecture. Processes are owned by a tmux server on the user's machine. A loopback-only bridge renders tabs onto in-world screens and forwards input. The client presents and accepts input but does not own the external agent jobs.

3.2 What survives what

Each Hermes tab is one named conversation (`hermes -p <profile> chat --tui -c omo:<workspace>:<tab>`), so reopening a screen brings back the same conversation [11]. Closing a screen, quitting Minecraft or restarting the bridge does not stop the agents. Restarting the computer does stop running jobs, but saved conversations can be resumed with a button. Idle Hermes tabs nap after 20 minutes unseen and wake when shown again; apart from that, "Nothing is ever re-run without you" [11].

Table 2. Documented lifecycle by event [11]; documented behaviour, not a survival test run for this paper. World layout is saved by Minecraft. Web-page state on a wall is not covered by this table,.

Event	Shells and running jobs	Hermes conversation
Close the view, walk away	Keep running	Keeps running
Quit or crash Minecraft	Keep running	Keeps running
Restart the bridge	Keep running	Keeps running
Hermes tab idle and unseen 20 min	—	Exits cleanly; resumes when shown
Host restart, or tmux server stops	**Lost**	Saved; tab shows INTERRUPTED; user presses Resume

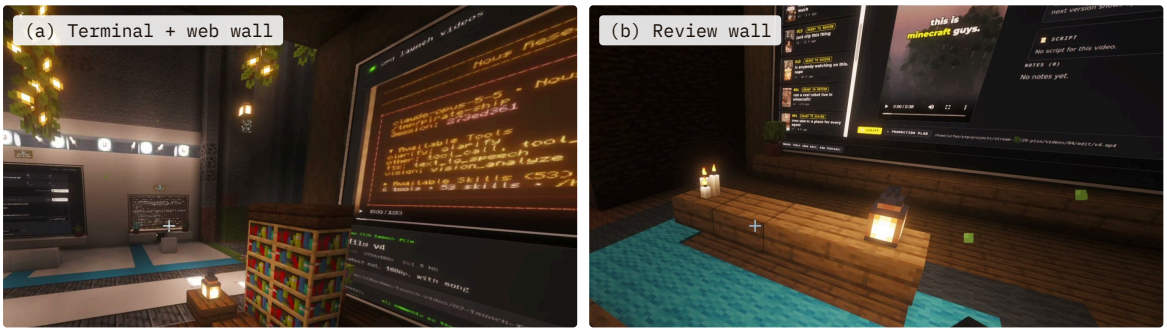


Figure 3. Agent work as objects in a place. (a) A Hermes session on a wall terminal (right) beside other agents' walls across the room. (b) An agent-built review page pinned on a wall: a queue of clips "ready to review" with a player and notes. Stills from the authors' live stream of 30 Oct 2026 (recordings `2026-10-30 05-26-05` and `23-31-03`), the day after the five days covered here, in the live world with real agent sessions. Frames are cropped to remove the face camera, stream label and game HUD.

3.3 Places built by agents

Places can be built and changed by coding agents from plain requests, and people also build by hand. Agents can also have bodies: a companion project brings a Hermes-driven player into the same world [13], and simulated robots appear in it. Bodies are outside the scope of this paper.

4 How We Looked Back

4.1 Who used it

From 25 to 29 October 2026 the builder (P1), a solo developer, simply used Omo Space as his main workspace while streaming to Twitch and YouTube. There was no study plan; the streams were his normal work. He ran a real content pipeline (ideas, references, recording, editing, publishing, analytics) with about one agent profile per stage, linked by his own scripts. A friend (P2), new to Hermes and on Windows, built his own world from the public repository with help from Claude Code and showed it on two streams. We did not inspect P2's code; his set-up time and how easy he found it are his own account (§5.7).

4.2 Data

We used seven main stream recordings and four short pickups, transcribed with Whisper (small), with a timestamp on every line. We split them into eight segments (Table 3), 29.05 hours in total. That figure is computed by `corpus_manifest.py` from the first and last timestamp in each analysed range. It includes silence, chat and replayed clips, so it is recorded time, not hours of distinct events. Two stretches were not analysed: the first 93 minutes of the 28 October part-2 recording (only partly transcribed) and 29 October after 3 h 44 min (not yet transcribed). The transcripts have no speaker labels, so who said what is inferred from context throughout. For P2 we also mark it at each quote.

Table 3. Corpus and extraction yield per segment. Counts are extracted records, not distinct events; repeated clips and scripted takes can yield several records.

Seg.	Recording (2026)	Hours	Theses	Quotes	Demos
T1	25 Oct, 02:08 (first stream)	2.80	17	59	18
T2	25 Oct, 18:56 + pickups (incl. 26 Oct)	1.99	15	60	17
T3	25 Oct, 23:41	3.90	17	55	14
T4	27 Oct, 18:03	3.77	17	59	25
T5	28 Oct, 01:36, 0:00–4:40	4.65	16	64	20
T6	28 Oct, 01:36, 4:40–9:20	4.67	18	54	23
T7	28 Oct, 11:40, 1:33–5:07	3.57	16	59	21
T8	29 Oct, 16:19, 0:00–3:44	3.70	17	59	18
Total		**29.05**	**133**	**469**	**156**

4.3 Real usage

The streams show what was said. Hermes also keeps its own token counters for every session, so we can say how much agent work actually ran (Table 4). Over the five days the builder's agents used about **7.1 billion tokens** in 43,157 model calls. On 28 October alone they used about **4.2 billion**; counting only sessions that started and ended that day it is still 3.75 billion. The day after the five days, 30 October, was busier again at 4.7 billion, with 20 agent profiles active. By 29 October twelve profiles were running, roughly one per pipeline stage.

Most of these tokens are context being re-read, not new text. From 24 October to 1 November, 95.7% were cache reads: each step of a long-running agent re-reads its whole conversation from the provider's prompt cache. Output was 66 million tokens (0.5%). Nearly all of it (99.3%) ran on Claude Opus 5.5. At list prices Hermes estimates the five days at about $3,400; no billed cost is recorded, so this is an equivalent, not what was paid. The counters cover all of the builder's Hermes work, including sessions that never appeared in the world, so they measure how much agent work he was running, not how much Omo Space caused. The script and per-day data are in the project's `usage/` folder.

Table 4. Real usage, from Hermes' own per-session token counters across all 45 local profile databases (read 1 Nov 2026). Days in AEDT; sessions that ran past midnight are split by time. The sessions column includes sub-agents and scheduled jobs. Cost is Hermes' estimate at published list prices, not an invoice.

Day	Tokens	API calls	Agent profiles	Sessions	List-price estimate
Sat 25 Oct	0.09 bn	970	3	33	$32
Sun 26 Oct	0.63 bn	5,004	9	137	$311
Mon 27 Oct	0.56 bn	3,989	9	94	$323
Tue 28 Oct	**4.17 bn**	25,386	11	537	$1,978
Wed 29 Oct	1.61 bn	7,808	12	159	$730
Five days	**7.07 bn**	**43,157**	—	—	**$3,374**
Thu 30 Oct (after)	4.67 bn	21,050	20	229	$2,094

4.4 Analysis

Reading. Each segment was read in full by a separate AI reader (Claude Opus 5.5). Each listed the claims made about the interface with 1–4 exact timestamped quotes each, the live demos, the failures and doubts, and guest reactions. All readers shared one brief. That brief asked what the builder was "trying to SAY/PROVE" and gave example claims, so it was looking for his intent, and may have nudged the readers (full prompts: Appendix A).

Grouping. An AI drafting assistant (also Claude) grouped the 133 segment-level claims into cross-segment themes. P1 checked the result against what he meant. No second person coded the material. The theme list, its rules and coverage counts are in Appendix C as an index; the findings below do not rest on the counts.

Labels on quotes. Each quote is tagged *live* (said while working), *take* (a scripted promo line recorded in the world) or *playback* (an edited clip replayed on stream). Takes show what P1 wanted to say, not what happened. Even live quotes are reports of what someone said, not observed outcomes. We worked from transcripts, not video.

Checks. A script confirmed that all 791 quotes the readers extracted appear in the transcripts within 3 minutes of their timestamps. A second, stricter script checks every quote in this paper against its own segment's transcript within 90 seconds. On v1.8 it found 30 timed matches and no failures, and it matched 14 literature quotes against the fetched sources (Appendix B). These checks show the words are there; they do not prove who said them or what happened. Key moments have not yet been checked against the audio.

4.5 Privacy and consent

The streams were public, and P1 is the system's author. Figures 3–5 are stream stills cropped so that no face camera, chat, viewer names or stream labels show. P2 has a pseudonym here, but the working files name him and his words can be found in the public recordings. We will confirm with him that he is happy to be quoted before this is shared.

5 What We Found

We read the material through four uses of place: find, resume, monitor and orchestrate. They are how we make sense of what happened, not proven effects. Table 5 sums them up; the sections after it give the evi-

dence. Then come the "web page with a terminal" pattern, places at every scale, and P2's world.

Table 5. The four uses of place at a glance. Each row: what it looked like, what the builders said helped, and the lesson we draw.

Use	What it looked like	What helped (reported)	Lesson
Find	Walk to the agent's place	Remembering places, not tab positions; about 25 sessions at once	Places help find work if they stay put and stay legible
Resume	Come back to where you left it	Agents still running after quitting the game	The place is an address; each kind of state needs its own guarantee
Monitor	Glance at lit screens and lamps	Screens "turn on" when an agent works (T8, 00:48:43, live)	Activity and "waiting for you" are different signals
Orchestrate	Move work from place to place	Pipeline stages visible as rooms	Space is not routing; hand-offs need to be explicit

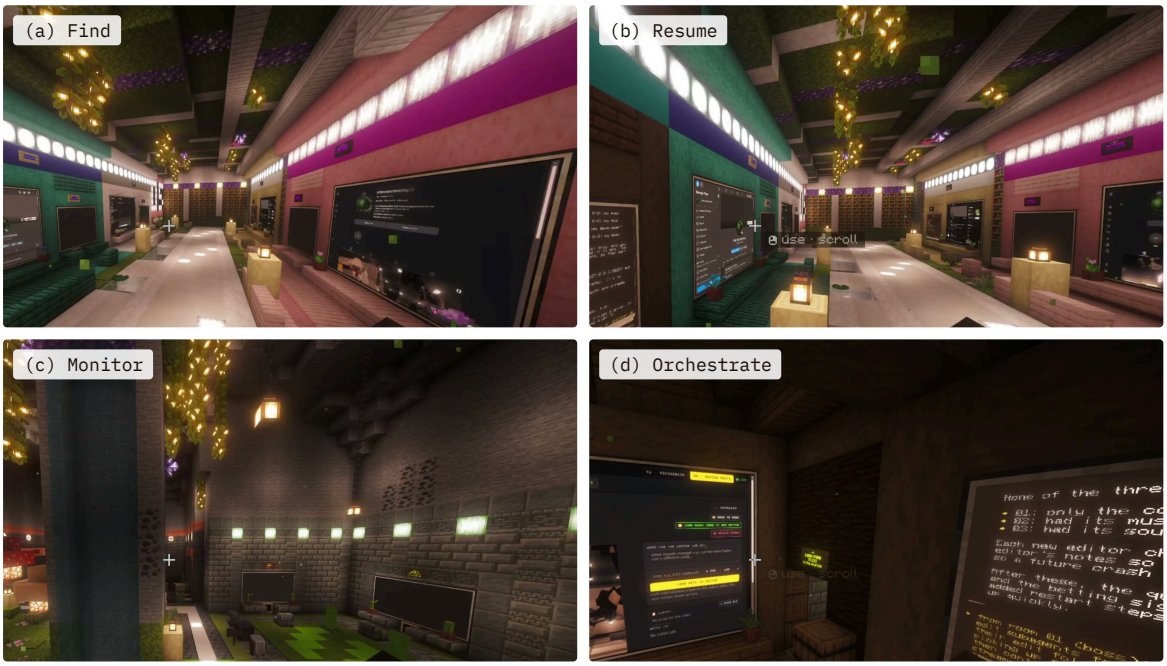


Figure 4. What each use of place looks like in the world. (a) *Find:* a corridor of places, each with its own screen. (b) *Resume:* a hall of agent places in the underground base he worked in every day. (c) *Monitor:* lit screens readable from the cavern entrance. (d) *Orchestrate:* the review stage of the video pipeline, with approve / send back / move to done controls, beside the editing agent's terminal. Stills from the authors' live stream of 30 Oct 2026 (recordings `2026-10-30 05-26-05` and `23-31-03`), the day after the five days covered here, in the live world with real agent sessions. Frames are cropped to remove the face camera, stream label and game HUD. The images illustrate layout; they do not show interaction outcomes.

5.1 Find: walking to the work

Every workspace sits in a fixed, named place, so you can remember where an agent is instead of which tab it was in. P1 put it most sharply in a scripted take:

> *"Nobody remembers which of the 12 terminal tabs are doing what but you can remember space and rooms… It's the oldest memory trick out there."* `[P1, T6, 07:14:29–07:14:46, take]`

Live, he named the problem the world was built to fix: "Getting lost with all these tabs and things" [T1, 01:13:45, live]. He reported about 25 sessions running at once (the logs show 537 sessions active on 28 October, counting sub-agents and scheduled jobs), "within the interface that we can conceptually understand" [T6, 07:37:22, live]. Whether the spatial-memory gains seen for documents [2] carry over to live agents is something we would like to measure (H1).

5.2 Resume: coming back to it

The agents run outside the game (Table 2), so leaving the game does not stop them, and the places are still there when you return. After quitting and reloading on the first night P1 said: "So we can close it enjoy our day And then Come back Boom the agents are still working right here" [T1, 02:12:55, live]. On 29 October he said agents had kept editing after he closed Minecraft, about 22 clips [T8, 00:06:30–00:08:38, live]. Separately, after turning the computer off and on, he found "everything's persistent" [T8, 00:11:55–00:11:59, live]. A restart keeps the world and the saved conversations but not running jobs (Table 2), so we read these as two separate moments, not as work done while the computer was off. He tied this to how it felt to start work: "I'm not dreading any pain / Or like context loss because I know anytime I want to work on the robotics project / I just fly down here" [T4, 02:21:42, live].

5.3 Monitor: glancing at lit screens

Lamps and lit screens let you read activity from across the room, much like calm technology [5]. P1's clearest lines are takes recorded in the Batcave: "because every light that you see in there is an agent still working" [T5, 00:36:26, take] and "Now you can see your whole team from the hallway." [T5, 00:39:00, take]. P2 described the same cue live, in his own world: "How I can tell when the task is like running because their screen will turn on" [T8, 00:48:43, live; P2, inferred]. He also said watching the team was more fun than "just looking at you know Claude you know just typing" [T8, 01:00:57, live; P2, inferred].

The lamps show who is busy. They do not yet show that an agent is waiting for an answer, and that is the next signal to build (H3).

5.4 Orchestrate: moving work between places

P1 laid out his content pipeline as a row of places, about one stage and one agent each: "We just go to that room and just start building / And when we're done, then we just leave that room" [T2, 01:04:14, live]. In a take he summed it up: "This video went through all five rooms before you saw it" [T5, 00:34:55, take]. By 29 October he wanted a place for each to-do item, "a repeatable pattern" [T8, 03:28:22, live]. In conversation, approving was described as a single in-world step: "once you press approve inside the game, it just gets posted" [T8, 00:56:28, live; speaker unclear].

The hand-offs between places are his own scripts and agent instructions, not a feature of the mod. Laying the pipeline out in space made its stages visible and easy to visit.

5.5 Web pages with a terminal attached

On the last day P1 described the idea in its most general form, apart from Minecraft:

> *"the concept of spatial interfaces with persistent webpages is easy to use and talk to and control. ... We're doing it inside Minecraft, but I believe there is not enough ... attention, these persistent webpages with a terminal attached to it. And with this, you can build whatever pipeline that you want."* `[P1, T8, 03:38:57–03:39:18, live]`

Pieced together from several moments, the pattern goes like this. You ask an agent at a station for a page for one job, such as a review queue, an analytics view or a publishing form. You pin it on a wall in the right place, check it, and approve or ask for changes through the terminal next to it. Then you come back to it later. The page does the job-specific work; the place gives it a fixed home next to related work. An ordinary browser tab could host the same page, and we did not separate the two effects. The in-world browser also had limits:

drop-down menus could not be selected [T8, 03:22:45, live]. The episode below shows one piece of it on the first night.

> **Episode A: leave and return (T1, 02:10:30–02:13:16; live; reports).** *Set-up.* P1 pins a web page on a wall beside working agents: "And then I get pins page to forever be here" [T1, 02:10:40]. *Action.* He closes the world: "I'm closing it entirely" [T1, 02:12:03]. *Result as reported.* On reloading: "Boom the agents are still working right here" [T1, 02:13:04], and "Web page is still here" [T1, 02:13:12]. *What it shows.* The agent sessions' survival matches the documented lifecycle (Table 2). The pinned page was there too.

5.6 Places at every scale

The places were not just rooms. P1 ran his content pipeline through a cave of five rooms. He wanted whole worlds for lines of work: "We can build a whole world for the paid ads" [T7, 02:48:38, live]. To show how personal it could get, he imagined a pirate ship holding your tax work [T6, 08:09:08, live]. He said agents did most of the building: "I didn't build any of this" [T6, 07:45:10, live]. He also liked building by hand sometimes: "I also think it's quite nice to just build with hand" [T1, 02:25:32, live]. The pattern is that people choose the shape and size of a place to fit the work, instead of the system choosing for them.

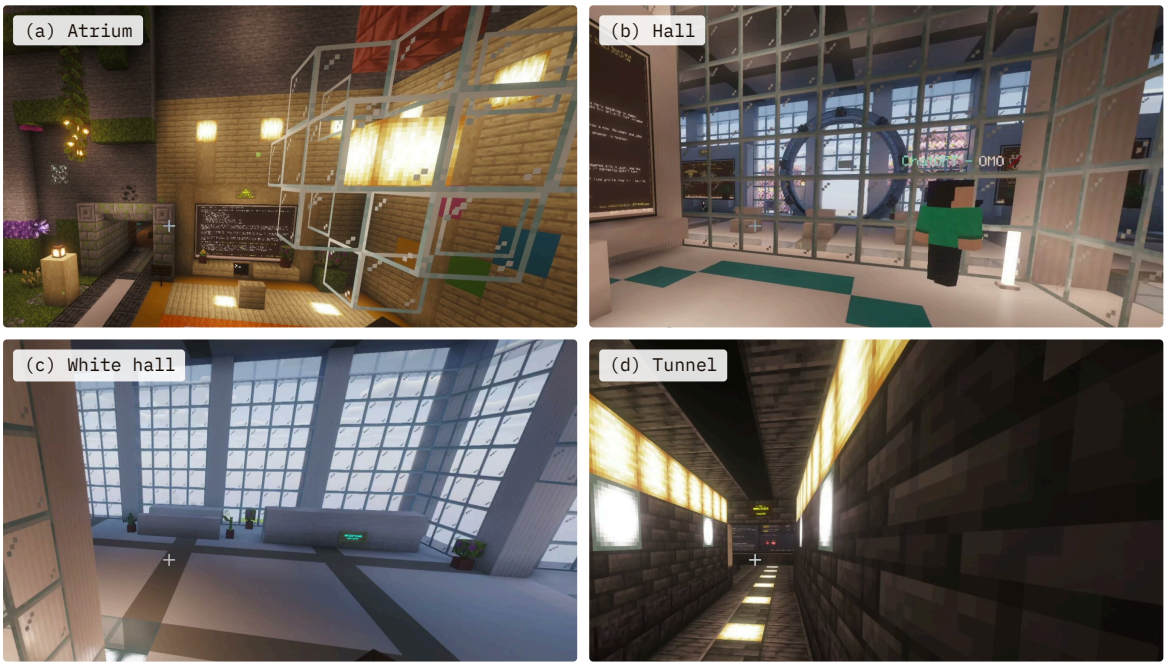


Figure 5. Places at several scales in the authors' world, from a single atrium to halls and the tunnels that connect them. In (b) an embodied agent (name tag "ChatGPT – …") stands in the hall. The authors report that these places were generated by agents from short requests; per-build prompts were not logged. Stills from the authors' live stream of 30 Oct 2026 (recordings `2026-10-30 05-26-05` and `23-31-03`), the day after the five days covered here, in the live world with real agent sessions. Frames are cropped to remove the face camera, stream label and game HUD.

5.7 A friend's world

This part comes from what was said during P2's screen-share, not from watching the video. P2 had never used Hermes before [T8, 00:59:38] and described about 24 hours of work [T8, 00:54:32] with little troubleshooting: "It just gave Claude the GitHub and then gave it a few prompts and then it just it built the world" [T8, 01:04:44, live; P2, inferred]. He showed an AI office with a manager agent, teams for web, social and IT, and a podium for talking to the whole team. Agents walked to their seats at a conference table, and a sports

arena showed live scores [T8, 00:28–01:13]. The manager, seating and podium were his own additions, not part of Omo Space; they show a second builder taking the idea his own way. Someone in the exchange described a meeting table where a message "will go to each of the four people's like memories and sessions" [T8, 01:12:04, live]; the transcript does not show who said it or whose table it was. The day before, P2 had called a page-only setup "claustrophobic" [T7, 04:16:50, live; P2, inferred].

6 Discussion

6.1 Design implications

1. **Give agent work a persistent, recognisable place,** while allowing several sessions per place, mirrored views and user-chosen scales.
2. **Separate address from state:** state the persistence of process, conversation, page and world separately (Table 2).
3. **Distinguish activity from a request for attention,** without colour as the only cue.
4. **Honest state:** never show progress that is not happening; re-run nothing on the user's behalf except documented idle resume.
5. **Let the work choose the shape:** places of any scale, built by agents or by hand.
6. **Make hand-offs between places explicit objects,** not a consequence of layout.

6.2 Why a game world, and why people liked it

Earlier 3D workspaces were designed by their makers [3]. A buildable world lets agents make the places from requests and lets other people build their own. That is what gave us places at every scale (§5.6) and P2's world (§5.7).

Both builders also chose where to work and said they liked being there. P1 described moving between reading and coding: "I can uh, read and then I can go to the other room … Do some coding or whatever and then go back to like reading and it's all together in one place in my mind" [T3, 02:01:07–02:01:11, live]. P2 said watching the team "makes this like AI coding and building stuff just more fun" [T8, 01:00:57, live; P2, inferred]. These are claims about how it felt, not wellbeing results.

6.3 Limitations

- **Just us so far.** One builder using his own tool, and one friend. Nothing was set up as a test, there is no comparison and nothing was measured. These are lessons from use, not proof that worlds beat tabs.
- **The builder is the author.** P1 built the system and was streaming, and some key lines are promo takes or replayed clips (§4.3). Studying your own system shows a lot but leans toward your own needs [16].
- **Analysis.** AI models did the reading and grouping, and P1 checked them. There was no second human coder.
- **Transcripts.** Whisper (small), no speaker labels. One line reads "I don't think Minecraft is superior" (T6, 08:33:45). It may be misheard, but until someone listens to the audio we leave it open.
- **Coverage.** About 29 hours analysed; two stretches were not (§4.2).
- **Pictures.** The figures show what the interface looks like, not results. Figures 3–5 are stills from 30 October, the day after the five days covered here; Figure 1 is a generated illustration.

7 What We Would Like to Measure Next

None of this has been tested yet. If we do test it, three of the four uses can be compared against ordinary tabs, and generated places can be looked at from logs (Table 6). The comparison must be fair. The tab setup gets the

same agents, the same persistence, good names, status badges and search, so any difference comes from space and not from better tools. Orchestration waits until hand-offs between places are real features.

Table 6. Proposed studies. H1–H3 are within-subjects and counterbalanced. Sample size is provisional (about 16–24) pending a pilot and power analysis. H4 is an observational log study.

Study	Prediction	Manipulation	Measure
H1 Find	Named sessions are located faster in the world than in tabs, and the gap widens with session count.	Interface (world, tabs) × sessions (12, 24)	Time to locate; wrong selections
H2 Resume	After a 24 h break, time to first useful action is shorter in the world.	Interface; 24 h delay; "useful action" defined in advance	Resumption lag; errors
H3 Monitor	With a proposed "waiting for you" lamp state (not yet built), a blocked session is noticed sooner than with equivalent tab badges when it is in view, but not when out of view unless escalation is added.	Interface × visibility (in view: same room / visible tab strip; out of view: other room / other workspace) × escalation (off, on); identical event schedule in both	Detection latency; misses (unnoticed within 5 min)
H4 Build (obs.)	Most one-prompt places are used without post-generation revisions.	None; log every build attempt (failed and abandoned included) started in days 1–30, each followed for 14 days (observation ends day 44)	Author acceptance; still in use after 14 days; revisions; independent usability rating against a pre-registered checklist

"Useful action" in H2 means the first edit, approval or instruction that moves the interrupted task forward. Participants practise each interface until they complete two trials unaided, and task versions are matched by session count and event schedule. In H4, a "one-prompt" place is one plain request with no follow-up, and any later edit the user asks for, by hand or by agent, counts as a revision. Some of the data already exists: bridge logs of sleep, resume and INTERRUPTED events, the Hermes session stores, and the recordings.

8 Conclusion

Chat windows were made for one conversation; agent work is now many conversations that keep going when you look away. We gave that work places in a world you can build, from a single room up to whole worlds. Its builder described finding, resuming, watching and routing agent work by moving through those places, and a friend took the idea his own way. Whether worlds help people more than windows is still an open question; §7 sketches how we would like to find out.

Data and Materials

The per-segment extractions, reader prompts, synthesis, theme matrix (`theme_matrix.py`), corpus manifest (`corpus_manifest.py`), quotation verifiers (`verify_quotes.py`, `verify_paper_quotes.py`), citation ledger with verbatim evidence and the review history are held in the project folder. They are not a public release: they contain identifying material about P2 and other speakers. A redacted evidence subset will be prepared before any release. Raw recordings are not released.

AI Use Disclosure

Language-model readers (Claude) performed first-pass extraction. A language-model assistant (Claude) proposed the cross-segment themes, coverage counts and the text of this draft, which P1 reviewed. An independent model (GPT-6 Astra) reviewed the draft against the evidence files and the full drafting history; its review is kept with the project materials. Stream quotations were checked by

script against transcripts (Appendix B), and literature quotations against fetched source text in the citation ledger. Those checks cover the existence of the words, not interpretation. The thesis-to-theme matrix in Appendix C was reconstructed by the drafting assistant after review.

References

[1] D. A. Henderson Jr. and S. K. Card. 1986. Rooms: The use of multiple virtual workspaces to reduce space contention in a window-based graphical user interface. *ACM Transactions on Graphics*. https://doi.org/10.1145/24054.24056

[2] G. Robertson, M. Czerwinski, K. Larson, D. Robbins, D. Thiel and M. van Dantzich. 1998. Data Mountain: Using spatial memory for document management. In *Proc. UIST '98*. ACM. https://www.microsoft.com/en-us/research/publication/data-mountain-using-spatial-memory-for-document-management

[3] G. Robertson et al. 2000. The Task Gallery: A 3D window manager. In *Proc. CHI '00*. ACM. https://doi.org/10.1145/332040.332482

[4] T. W. Malone. 1983. How do people organize their desks? Implications for the design of office information systems. *ACM Transactions on Office Information Systems*. https://doi.org/10.1145/357423.357430

[5] M. Weiser and J. S. Brown. 1995. Designing calm technology. Xerox PARC, 21 December 1995. https://calmtech.com/papers/designing-calm-technology

[6] M. Czerwinski, E. Horvitz and S. Wilhite. 2004. A diary study of task switching and interruptions. In *Proc. CHI '04*. ACM. https://doi.org/10.1145/985692.985715

[7] R. M. Ratwani and J. G. Trafton. 2006. Now, where was I? Examining the perceptual processes while resuming an interrupted task. In *Proc. 28th Annual Meeting of the Cognitive Science Society*. https://escholarship.org/uc/item/7br305k1

[8] G. Wang et al. 2023. Voyager: An open-ended embodied agent with large language models. arXiv:2305.16291.

[9] J. S. Park et al. 2023. Generative agents: Interactive simulacra of human behavior. arXiv:2304.03442.

[10] I. White, K. Nottingham, A. Maniar, M. Robinson, H. Lillemark, M. Maheshwari, L. Qin and P. Ammanabrolu. 2025. Collaborating action by action: A multi-agent LLM framework for embodied reasoning. arXiv:2504.17950.

[11] Omo Research. 2026. Omo Space documentation: Agents and terminals. docs/AGENTS-AND-TERMINALS.md, revision 75d4677. https://github.com/omoresearch/omo-space

[12] Omo Research. 2026. Omo Space changelog. CHANGELOG.md, revision 75d4677. https://github.com/omoresearch/omo-space

[13] Omo Research. 2026. Omo, brought to life (v0.1.0). Source repository, README.md (local repository; revision not pinned).

[14] This study. 2026. Stream analysis: per-segment extractions and synthesis of seven recordings and pickups, 25–29 October 2026.

[15] V. Braun and V. Clarke. 2006. Using thematic analysis in psychology. *Qualitative Research in Psychology* 3(2), 77–101. https://doi.org/10.1191/1478088706qp063oa

[16] C. Neustaedter and P. Sengers. 2012. Autobiographical design in HCI research: Designing and learning through use-it-yourself. In *Proc. DIS '12*. ACM. https://doi.org/10.1145/2317956.2318034

[17] Z. Xiao, X. Yuan, Q. V. Liao, R. Abdelghani and P.-Y. Oudeyer. 2023. Supporting qualitative analysis with large language models: Combining codebook with GPT-3 for deductive coding. In *IUI '23 Companion*. arXiv:2304.10548.

[18] C. Ellis, T. E. Adams and A. P. Bochner. 2011. Autoethnography: An overview. *Historical Social Research* 36(4), 273–290. https://doi.org/10.12759/hsr.36.2011.4.273-290

[19] T. Long, W. Shi, H. Mozannar, M. Murad and R. Hosn. 2026. ParallelPilot: Supporting coordination and monitoring in parallel AI coding. arXiv:2609.33113. https://arxiv.org/abs/2609.33113

[20] X. Zhao, J. Sohn, Q. Zheng and M. Moor. 2026. AgentGUI: An interface for observing and steering long-running AI agents. arXiv:2607.26300. https://arxiv.org/abs/2607.26300

[21] H. Mozannar, G. Bansal, C. Tan, A. Fourney, V. Dibia et al. 2025. Magentic-UI: Towards human-in-the-loop agentic systems. arXiv:2507.22358. https://arxiv.org/abs/2507.22358

[22] Y. Leviathan, D. Valevski, M. Kalman et al. 2026. Generative UI: LLMs are effective UI generators. arXiv:2604.09577. https://arxiv.org/abs/2604.09577

[23] C. C. Marshall, F. M. Shipman III and J. H. Coombs. 1994. VIKI: Spatial hypertext supporting emergent structure. In *Proc. ECHT '94*, 13–23. ACM. https://doi.org/10.1145/192757.192759

Appendix A. Reader Prompts

The eight reader prompts (one shared brief plus a per-segment assignment) are archived verbatim in `stream-analysis/READER-PROMPTS.json`, recovered from the session store. The shared brief asks the reader to read the assigned range completely and extract: (1) theses, i.e. every claim the builder is "trying to demonstrate/prove about the interface", with examples given; (2) 1–4 verbatim quotations per thesis with line timestamps; (3) live demonstrations; (4) counter-evidence and friction, including sceptical guest reactions; (5) guest and viewer reactions; and (6) memorable phrasings. It asks for honesty when a chunk is unrelated and says "Never invent quotes or timestamps". These are instructions; compliance was checked only for quotation existence (Appendix B). The example theses in the brief may have primed the readers.

Appendix B. Quotation Verification

Corpus check. `verify_quotes.py` normalises each quotation record from the readers' theses, friction and phrasings (lowercase, punctuation stripped except apostrophes, whitespace collapsed, split on ellipses). It requires the fragments in order within ±180 s of the stated time. Result, first run 29 Oct 2026 and re-run by the reviewer on 30 Oct: 791 of 791 matched. Limits: T2 pickups are pooled, records without a parsable time are searched without a window, and speakers and live/replay status are not checked.
Paper check. `verify_paper_quotes.py` detects selected main-text quotations (curly-quoted spans before the References) and checks their text near the supplied locators. It requires every fragment, split on ellipses and " / ", to occur in order in the stated segment's own transcript, between 90 s before the earliest and 90 s after the latest time in the locator. T2 pickups are resolved by file name. A listed set of scare-quoted terms in our own voice is skipped. Literature excerpts are matched against the pooled fetched source texts and cited local documentation. This is not a check of attribution to a specific reference; that was done by hand and is recorded with evidence in `ledger.json`. Result on v1.8 is recorded in the study README.

Appendix C. Thesis-to-Theme Matrix (reconstructed)

Files. `theme_matrix.py` holds the rules, the mapping and the change log, and checks without writing. `--write` regenerates `stream-analysis/THEME-MATRIX.json`; the input is restricted to the eight segment files. Current matrix: 133 enumerated theses; 117 linked to at least one theme; 16 unlinked; 137 links. The first-pass reconstruction is kept as `theme_matrix_v1.py`.
Changes in the second pass. T8-8 (one-prompt delegation of websites, ads and clips) was removed from C7 because no place is built. T8-5 (P2's world "built" by Claude from the repository) was added to C7. T6 T15 (an observation deck and feeling safe) was removed from C5 because no agent state is read. T3-11 (gamified, fun work) was moved from C12 to C13. T1-5 was added to C5, because C5 includes proposals. Five theses were added to C13. Net effect against the first pass: C5 8 → 7, C12 5 → 4, C13 new at 5; C7 unchanged at 7, now resting on T8-5 rather than T8-8.
Worked positive case. T6 thesis T2 (strength 5), "Rooms, not tabs: spatial memory … beats remembering which of N terminal tabs does what", with quotation [T6, 07:14:29, take], meets the C1 rule.
Worked boundary case. T1-5 (strength 4), "Reading text in chat boxes causes fatigue … wants visual, glanceable or voice output". It meets C4 (reading fatigue) and, because C5 counts proposals, C5. Under a use-only reading it would leave C5; that would not change C5's count, because T1 has other qualifying theses.
Fragility. The script lists every cell where dropping one link lowers a theme's count; for example, C7 in T8 rests only on T8-5, and C10 in T1 only on T1-10. The negative case in the corpus ("I don't think Minecraft is superior", T6 08:33:45) is a separate, unresolved ASR question, not part of this matrix.

Excluding C11 (a product claim), seven themes appear in at least seven of eight segments; four of them sit at exactly 7/8 and depend on single links in at least one segment.

Table C1. Cross-segment themes. Coverage = segments with a mapped thesis at centrality ≥3 (n = 8; reconstructed matrix, Appendix C). Rows are listed by theme identifier. C11 is a product claim, C12–C13 concern audience and experience. Rules for each theme are in `theme_matrix.py`.

#	Theme (as reported by participants)	Cov.	Representative quotation
C1	Locating work by place rather than by tab	8/8	*"you can remember space and rooms"* (T6, 07:14:29, take)
C2	The place holds the work; return rather than reopen	8/8	*"Boom the agents are still working right here"* (T1, 02:13:04, live)
C3	An interface, not a chat window	8/8	*"It's not a chat window, it's an interface."* (T1, 00:38:50, live)
C4	Cognitive fit and ease of returning	7/8	*"I'm not dreading any pain / Or like context loss"* (T4, 02:21:42, live)
C5	Agent state legible from the surroundings	7/8	*"see your whole team from the hallway"* (T5, 00:39:00, take)
C6	Workflows laid out as places to move through	7/8	*"we just walk it through the pipeline"* (T2, 01:38:37, live)
C7	Places generated from requests	7/8	*"I didn't build any of this"* (T6, 07:45:10, live)
C8	Real tools, not a toy	6/8	*"They're real agents."* (T5, 00:36:58, take)
C9	Embodied co-inhabitants (voice, robots)	6/8	*"Yo ducks ducks race to my position"* (T5, 03:02:26, live)
C10	Transfer to other people and shared worlds	5/8	*"It just gave Claude the GitHub"* (T8, 01:04:44, live)
C11	Business throughput (product claim)	7/8	—
C12	Creativity and access for non-coders	4/8	—
C13	Enjoyment and inhabitation	5/8	*"It makes it makes it more fun"* (T8, 01:00:57, live; P2, inferred)